

思考模式

DeepSeek 模型支持思考模式：在输出最终回答之前，模型会先输出一段思维链内容，以提升最终答案的准确性。

思考模式开关与思考强度控制

	控制参数（OpenAI 格式）	控制参数（Anthropic 格式）
思考模式开关 (1)	<code>{"thinking": {"type": "enabled/disabled"}}</code>	
思考强度控制 (2)(3)	<code>{"reasoning_effort": "high/max"}</code>	<code>{"output_config": {"effort": "high/max"}}</code>

- (1) 默认思考开关为 `enabled`
- (2) 思考模式下，对普通请求，默认 `effort` 为 `high`；对一些复杂 Agent 类请求（如 Claude Code、OpenCode），`effort` 自动设置为 `max`
- (3) 思考模式下，出于兼容考虑 `low`、`medium` 会映射为 `high`，`xhigh` 会映射为 `max`

您在使用 OpenAI SDK 设置 `thinking` 参数时，需要将 `thinking` 参数传入 `extra_body` 中：

```
response = client.chat.completions.create(  
    model="deepseek-v4-pro",  
    # ...  
    reasoning_effort="high",  
    extra_body={"thinking": {"type": "enabled"}}  
)
```

输入输出参数

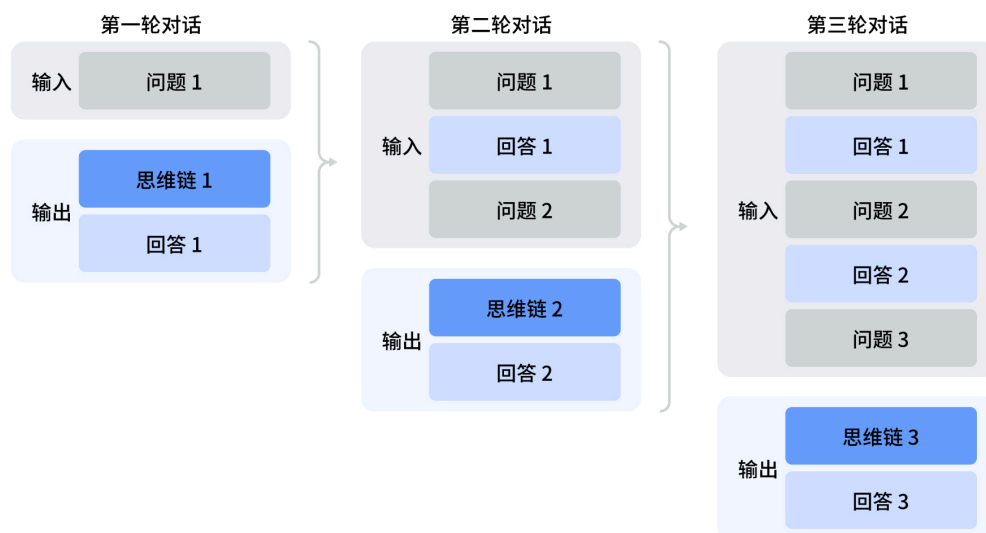
思考模式不支持 `temperature`、`top_p`、`presence_penalty`、`frequency_penalty` 参数。请注意，为了兼容已有软件，设置参数不会报错，但也不会生效。

在思考模式下，思维链内容通过 `reasoning_content` 参数返回，与 `content` 同级。在后续的轮次的拼接中，可以选择性地返回 `reasoning_content` 给 API：

- 在两个 `user` 消息之间，如果模型**未进行工具调用**，则中间 `assistant` 的 `reasoning_content` 无需参与上下文拼接，在后续轮次中将其传入 API 会被忽略。详见[多轮对话拼接](#)。
- 在两个 `user` 消息之间，如果模型**进行了工具调用**，则中间 `assistant` 的 `reasoning_content` 需参与上下文拼接，在后续所有 `user` 交互轮次中必须回传给 **API**。详见[工具调用](#)。

多轮对话拼接

在每一轮对话过程中，模型会输出思维链内容（`reasoning_content`）和最终回答（`content`）。如果没有工具调用，则在下一轮对话中，之前轮输出的思维链内容不会被拼接到上下文中，如下图所示：



样例代码

下面的代码以 Python 语言为例，展示了如何访问思维链和最终回答，以及如何在多轮对话中进行上下文拼接。

[非流式](#) [流式](#)

```
from openai import OpenAI
client = OpenAI(api_key="<DeepSeek API Key>",
base_url="https://api.deepseek.com")
```

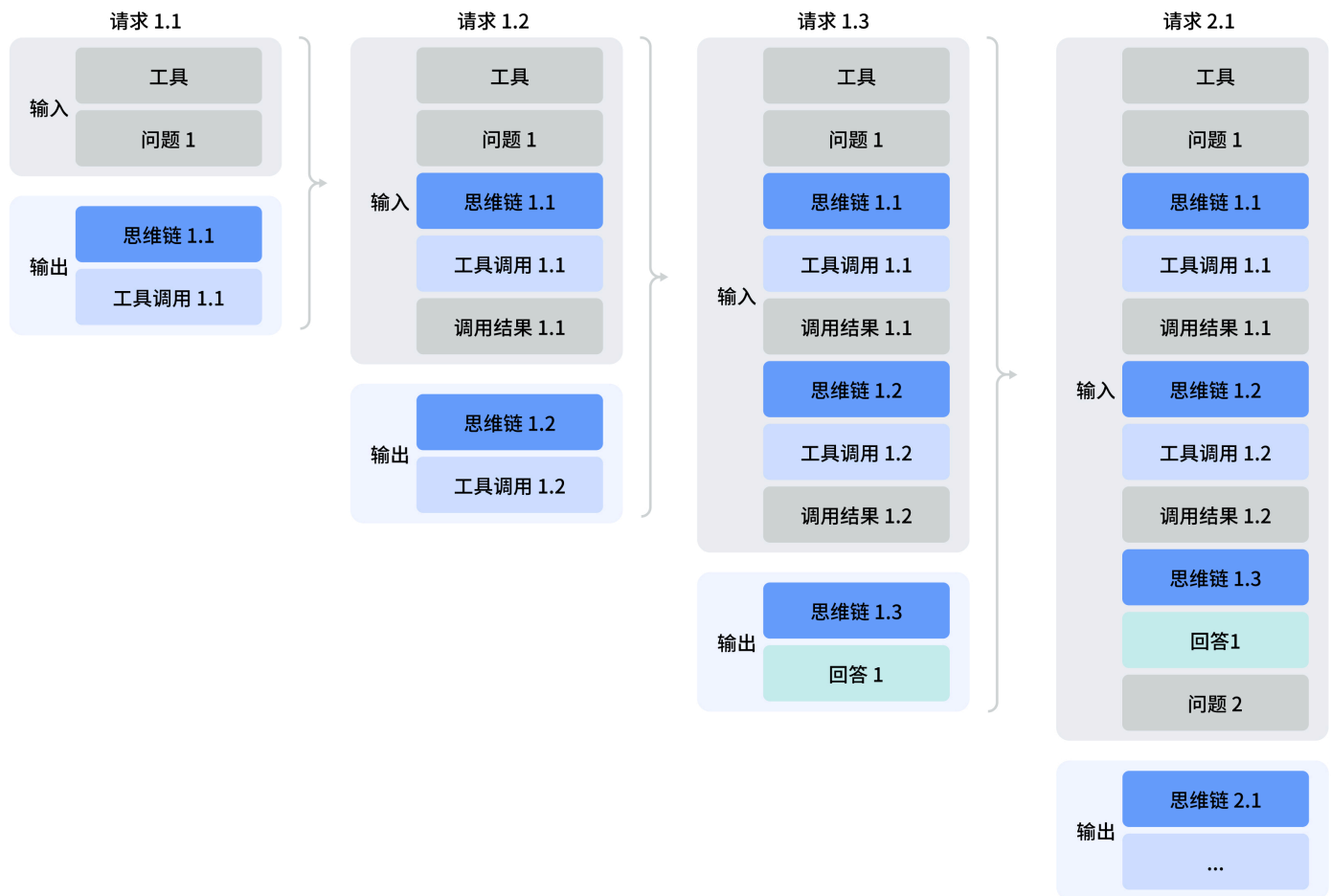
```
# Turn 1
messages = [{"role": "user", "content": "9.11 and 9.8, which is greater?"}]
response = client.chat.completions.create(
    model="deepseek-v4-pro",
    messages=messages,
    reasoning_effort="high"
    extra_body={"thinking": {"type": "enabled"}}},
)

reasoning_content = response.choices[0].message.reasoning_content
content = response.choices[0].message.content

# Turn 2
# The reasoning_content will be ignored by the API
messages.append(response.choices[0].message)
messages.append({'role': 'user', 'content': "How many Rs are there in the word 'strawberry'?"})
response = client.chat.completions.create(
    model="deepseek-v4-pro",
    messages=messages,
    reasoning_effort="high"
    extra_body={"thinking": {"type": "enabled"}}},
)
# ...
```

工具调用

DeepSeek 模型的思考模式支持工具调用功能。模型在输出最终答案之前，可以进行多轮的思考与工具调用，以提升答案的质量。其调用模式如下图所示：



请注意，区别于思考模式下的未进行工具调用的轮次，进行了工具调用的轮次，在后续所有请求中，必须完整回传 `reasoning_content` 给 API。

若您的代码中未正确回传 `reasoning_content`，API 会返回 400 报错。正确回传方法请您参考下面的样例代码。

样例代码

下面是一个简单的在思考模式下进行工具调用的样例代码：

```
import os
import json
from openai import OpenAI
from datetime import datetime

# The definition of the tools
tools = [
    {
        "type": "function",
        "function": {
```

```

        "name": "get_date",
        "description": "Get the current date",
        "parameters": { "type": "object", "properties": {} },
    },
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get weather of a location, the user should
supply the location and date.",
            "parameters": {
                "type": "object",
                "properties": {
                    "location": { "type": "string", "description": "The
city name" },
                    "date": { "type": "string", "description": "The date
in format YYYY-mm-dd" },
                },
                "required": ["location", "date"]
            },
        }
    },
]

```

The mocked version of the tool calls

```

def get_date_mock():
    return datetime.now().strftime("%Y-%m-%d")

def get_weather_mock(location, date):
    return "Cloudy 7~13°C"

TOOL_CALL_MAP = {
    "get_date": get_date_mock,
    "get_weather": get_weather_mock
}

def run_turn(turn, messages):
    sub_turn = 1
    while True:
        response = client.chat.completions.create(
            model='deepseek-v4-pro',
            messages=messages,
            tools=tools,

```

```

        reasoning_effort="high",
        extra_body={ "thinking": { "type": "enabled" } },
    )
    messages.append(response.choices[0].message)
    reasoning_content = response.choices[0].message.reasoning_content
    content = response.choices[0].message.content
    tool_calls = response.choices[0].message.tool_calls
    print(f"Turn {turn}.
{sub_turn}\n{reasoning_content=}\n{content=}\n{tool_calls=}")
    # If there is no tool calls, then the model should get a final
    answer and we need to stop the loop
    if tool_calls is None:
        break
    for tool in tool_calls:
        tool_function = TOOL_CALL_MAP[tool.function.name]
        tool_result =
tool_function(**json.loads(tool.function.arguments))
        print(f"tool result for {tool.function.name}:
{tool_result}\n")
        messages.append({
            "role": "tool",
            "tool_call_id": tool.id,
            "content": tool_result,
        })
        sub_turn += 1
    print()

client = OpenAI(
    api_key=os.environ.get('DEEPSEEK_API_KEY'),
    base_url=os.environ.get('DEEPSEEK_BASE_URL'),
)

# The user starts a question
turn = 1
messages = [{
    "role": "user",
    "content": "How's the weather in Hangzhou Tomorrow"
}]
run_turn(turn, messages)

# The user starts a new question
turn = 2
messages.append({
    "role": "user",

```

```
    "content": "How's the weather in Guangzhou Tomorrow"
  })
  run_turn(turn, messages)
```

在 Turn 1 的每个子请求中，都携带了该 Turn 下产生的 `reasoning_content` 给 API，从而让模型继续之前的思考。`response.choices[0].message` 携带了 `assistant` 消息的所有必要字段，包括 `content`、`reasoning_content`、`tool_calls`。简单起见，可以直接用如下代码将消息 append 到 `messages` 结尾：

```
messages.append(response.choices[0].message)
```

这行代码等价于：

```
messages.append({
    'role': 'assistant',
    'content': response.choices[0].message.content,
    'reasoning_content': response.choices[0].message.reasoning_content,
    'tool_calls': response.choices[0].message.tool_calls,
})
```

且在 Turn 2 的请求中，我们仍然携带着 Turn1 所产生的 `reasoning_content` 给 API。

该代码的样例输出如下：

```
Turn 1.1
reasoning_content="The user is asking about the weather in Hangzhou tomorrow
I need to get tomorrow's date first, then call the weather function."
content="Let me check tomorrow's weather in Hangzhou for you. First, let me
get tomorrow's date."
tool_calls=
[ChatCompletionMessageFunctionToolCall(id='call_00_kw66qNnNto11bSfJVIIdlV50o'
function=Function(arguments='{}', name='get_date'), type='function',
index=0)]
tool result for get_date: 2026-04-19

Turn 1.2
reasoning_content="Today is 2026-04-19, so tomorrow is 2026-04-20. Now I'll
call the weather function for Hangzhou."
content=''
```

```
tool_calls=[ChatCompletionMessageFunctionToolCall(id='call_00_H2SCW6136vWJGq9SQlBuhVt4', function=Function(arguments='{"location": "Hangzhou", "date": "2026-04-20"}', name='get_weather'), type='function', index=0)]
tool result for get_weather: Cloudy 7~13°C
```

Turn 1.3

```
reasoning_content='The weather result is in. Let me share this with the user.'
```

```
content="Here's the weather forecast for **Hangzhou tomorrow (April 20, 2026)**:\n\n- 🌤️ **Condition:** Cloudy \n- 🌡️ **Temperature:** 7°C ~ 13°C (45°F ~ 55°F)\n\nIt'll be on the cooler side, so you might want to bring a light jacket if you're heading out! Let me know if you need anything else."
tool_calls=None
```

Turn 2.1

```
reasoning_content='The user is asking about the weather in Guangzhou tomorrow. Today is 2026-04-19, so tomorrow is 2026-04-20. I can directly call the weather function.'
```

```
content=''
```

```
tool_calls=[
```

```
ChatCompletionMessageFunctionToolCall(id='call_00_8URkLt5NjmNkVKhDmMcNq9Mo', function=Function(arguments='{"location": "Guangzhou", "date": "2026-04-20"}', name='get_weather'), type='function', index=0)]
```

```
tool result for get_weather: Cloudy 7~13°C
```

Turn 2.2

```
reasoning_content='The weather result for Guangzhou is the same as Hangzhou. Let me share this with the user.'
```

```
content="Here's the weather forecast for **Guangzhou tomorrow (April 20, 2026)**:\n\n- 🌤️ **Condition:** Cloudy \n- 🌡️ **Temperature:** 7°C ~ 13°C (45°F ~ 55°F)\n\nIt'll be cool and cloudy, so a light jacket would be a good idea if you're going out. Let me know if there's anything else you'd like to know!"
```

```
tool_calls=None
```